Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №2

«Анализ и устранение уязвимости на примере реального CVE с использованием Vulhub»

Капарулин Тимофей
Иванович

Группа: P3408

2026

Содержание

1. **Название выбранной уязвимости и краткое ее описание**
2. **Последовательность действий по воспроизведению уязвимости**
3. **Анализ root cause**
4. **Описание примененного исправления**
5. **Доказательство устранения уязвимости**

Название выбранной уязвимости и краткое ее описание

CVE-2019-9053 - это уязвимость неаутентифицированного слепого SQL-инъекционирования на основе времени

Проблема обнаружена в модуле News, уязвимость проявляется при обработке параметра m1_idlist. Злоумышленник может отправить специально сформированный запрос, который заставит базу данных выполнить SQL-инъекцию. Поскольку результат запроса не возвращается в виде обычного текста в ответе сервера, атака является слепой.

Атакующий использует задержку ответа сервера: если проверяемое условие в SQL-запросе истинно, база данных выполняет функцию sleep(1), и запрос занимает одну секунду. Если условие ложно, ответ приходит мгновенно. Таким образом, анализируя время отклика, атакующий побитово извлекает данные из базы

Последовательность действий по воспроизведению уязвимости

1) Разворачиваем CMS Made Simple версии 2.2.9.1 через команду

```
docker compose up -d
```

2) Для настройки CMS переходим по адресу http://127.0.0.1/install.php и вводим такие значения:

- database address - db
- database name - cmsms
- username - root
- password - root

Database Information

CMS Made Simple stores a great deal of data in the database. A database connection is mandatory. Additionally, the user credentials you supply should have ALL PRIVILEGES on the specified database to allow creating, dropping and modifying tables, indexes and views.

Database Hostname	db *
Database Name	cmsms *
User name	root *
Password	•••• *

Server Timezone

The time zone information is needed for time calculations and time/date displays. Please select the server timezone

UTC

Next →

Рисунок 1 — Данные для настройки CMS

1. **Welcome** ✓
   Start the installation or upgrade process
2. **Detect Existing Software** ✓
   Analyze destination directory to find existing software
3. **Compatibility Tests** ✓
   Check to make sure everything is OK to install the CMSMS core
4. **Configuration Info** ✓
   For new installs, and freshen operation, enter basic configuration info
5. **Admin Account Info** ✓
   For new installs, enter Admin account info
6. **Site Settings** ✓
   For new installs enter some basic site details
7. **Files** ✓
   Extract files
8. **Database work** ✓
   Create or update the database schema, set initial events, permissions, user accounts, templates, stylesheets and content
9. **Finish**
   Install and/or Upgrade modules as necessary, write the config file, and clean up.

Creating a new CMSMS 2.2.9.1 website

Step 9 - Finish

Installation Directory:
/var/www/html

Create temporary directories

Install available modules

Send Admin login credentials email

Error sending mail: Could not instantiate mail function.

Cleared server cache

We are done! You can now visit your website or login to the CMSMS admin panel.

Warning For security reasons it is important that you remove the installation assistant from your browseable website as soon as you have verified that the operation has succeeded.

Share your experience with your friends.

GOOGLE+ FACEBOOK TWITTER LINKEDIN

Рисунок 2 — Завершение настройки CMS

3) Скачиваем скрипт с CMS Made Simple < 2.2.10 - SQL Injection

4) Запускаем скрипт

```
python poc.py -u http://127.0.0.1
```

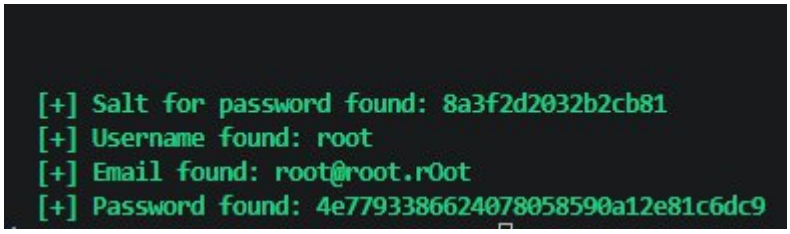


Рисунок 3 — Результат выполнения скрипта до фикса

Анализ root cause

Уязвимость возникла из-за того, что модуль News в CMS Made Simple (версии 2.2.9 и ниже) принимал параметр m1_idlist из HTTP-запроса и подставлял его в SQL-запрос. Предполагалось, что этот параметр будет содержать только список идентификаторов новостей - то есть цифры, разделённые запятыми. Разработчики предприняли попытку валидации: каждый элемент разбивался через explode, приводился к целому числу (int), а нечисловые значения отбрасывались через unset. Однако эта проверка содержала логическую ошибку, которая делала её обходимой.

В уязвимом коде использовался цикл for с условием $i < count($tmp) и оператором unset($tmp[$i]) для удаления нечисловых элементов:

```
$tmp = explode(',', $idlist);
for( $i = 0; $i < count($tmp); $i++ ) {
   $tmp[$i] = (int)$tmp[$i];
   if( $tmp[$i] < 1 ) unset($tmp[$i]);
}
```

Условие count($tmp) пересчитывается на каждой итерации, а unset() уменьшает длину массива. Из-за этого счётчик $i и фактическое количество элементов расходятся: как только первые элементы удаляются, цикл завершается раньше, чем обработаны все данные. Часть элементов (в том числе содержащих SQL-код) остаётся в массиве нетронутой и попадает в запрос.

Поскольку SQL-запрос формировался путём конкатенации строк, проскочившие фильтр данные попадали непосредственно в текст запроса и начинали интерпретироваться базой данных как часть SQL-логики. Это позволило провести слепую SQL-инъекцию на основе времени (time-based blind SQL injection). Атакующий внедрял в параметр условия с вызовом

SLEEP() и по задержке ответа сервера определял, истинно условие или нет. Так, побайтово, можно было извлечь соль, имя пользователя, email и хеш пароля администратора из таблиц cms_users и cms_siteprefs.

Уязвимость была неаутентифицированной, потому что точка входа moduleinterface.php доступна без входа в административную панель. Корневая причина - отсутствие параметризованных запросов и строгой проверки типа входных данных. Иными словами, приложение доверяло пользовательскому вводу и использовало его при построении SQL-запроса без приведения к ожидаемому типу (например, к целому числу) и без экранирования.

Описание примененного исправления

В уязвимом коде параметр idlist из пользовательского запроса напрямую подставлялся в SQL-запрос без какой-либо проверки. Это позволяло злоумышленнику внедрять произвольные SQL-конструкции. Исправление изменяет способ обработки этого параметра: вместо прямой конкатенации выполняется строгая валидация с приведением к целочисленному типу.

Что именно было изменено:

1. Входная строка $params['idlist'] разбивается по запятым на массив с помощью explode(',', ...)
2. Каждый элемент массива очищается от пробелов по краям функцией trim()
3. С помощью ctype_digit() проверяется, что элемент состоит только из цифр (0–9). Это ключевой момент: любые другие символы (буквы, кавычки, скобки, операторы SQL) приводят к тому, что элемент отбрасывается
4. Если проверка пройдена, значение принудительно приводится к целому числу (int). Это дополнительная страховка: даже если в строке остались какие-то нецифровые символы, приведение к целому типу их отсекает
5. Все безопасные целые числа собираются в массив $safe_ids, который затем объединяется обратно в строку через implode(',', ...)
6. Только эта итоговая строка, состоящая исключительно из цифр и запятых, подставляется в SQL-запрос в конструкции IN (...)

Конкретные изменения в коде:

```
// Исходный уязвимый блок
if( isset($params['idlist']) ) {
    $idlist = $params['idlist'];
    if( is_string($idlist) ) {
        $tmp = explode(',',$idlist);
        for( $i = 0; $i < count($tmp); $i++ ) {
            $tmp[$i] = (int)$tmp[$i];
            if( $tmp[$i] < 1 ) unset($tmp[$i]);
        }
        $idlist = array_unique($tmp);
        $query1 .= ' (mn.news_id IN ('.implode(',',$idlist).')) AND ';
    }
}

// Исправленная версия кода
if( isset($params['idlist']) ) {
    $id_array = explode(',', $params['idlist']);
    $safe_ids = [];
    foreach ($id_array as $id) {
        $id = trim($id);
        if (ctype_digit($id)) {
            $safe_ids[] = (int)$id;
        }
    }
    $query1 .= ' (mn.news_id IN ('.implode(',',$safe_ids).')) AND ';
}
```

Почему этот метод эффективен:

1. Принцип белого списка: разрешаются только заведомо безопасные - цифры. Всё остальное отбрасывается. Это исключает возможность обхода фильтра с помощью нестандартных кодировок или неизвестных символов

2. Приведение к целому числу: Даже если бы ctype_digit() по какой-то причине пропустила нецифровой символ, (int) гарантированно преобразует значение в число, отбросив всё лишнее. Например, (int)"1; DROP TABLE" вернёт 1, а (int)"abc" вернёт 0

3. Метод не зависит от конфигурации СУБД, не требует экранирования и не ломает логику приложения. Если злоумышленник передаст вредоносную строку, она просто не пройдёт валидацию, и в запрос не попадёт ничего опасного

Доказательство устранения уязвимости

После внесенных изменений в исходный код запускаем скрипт эксплойта:

```
python poc.py -u http://127.0.0.1
```

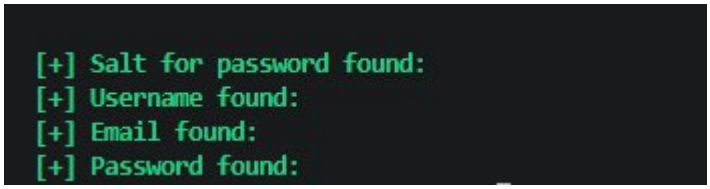


Рисунок 4 — Результат выполнения скрипта после фикса